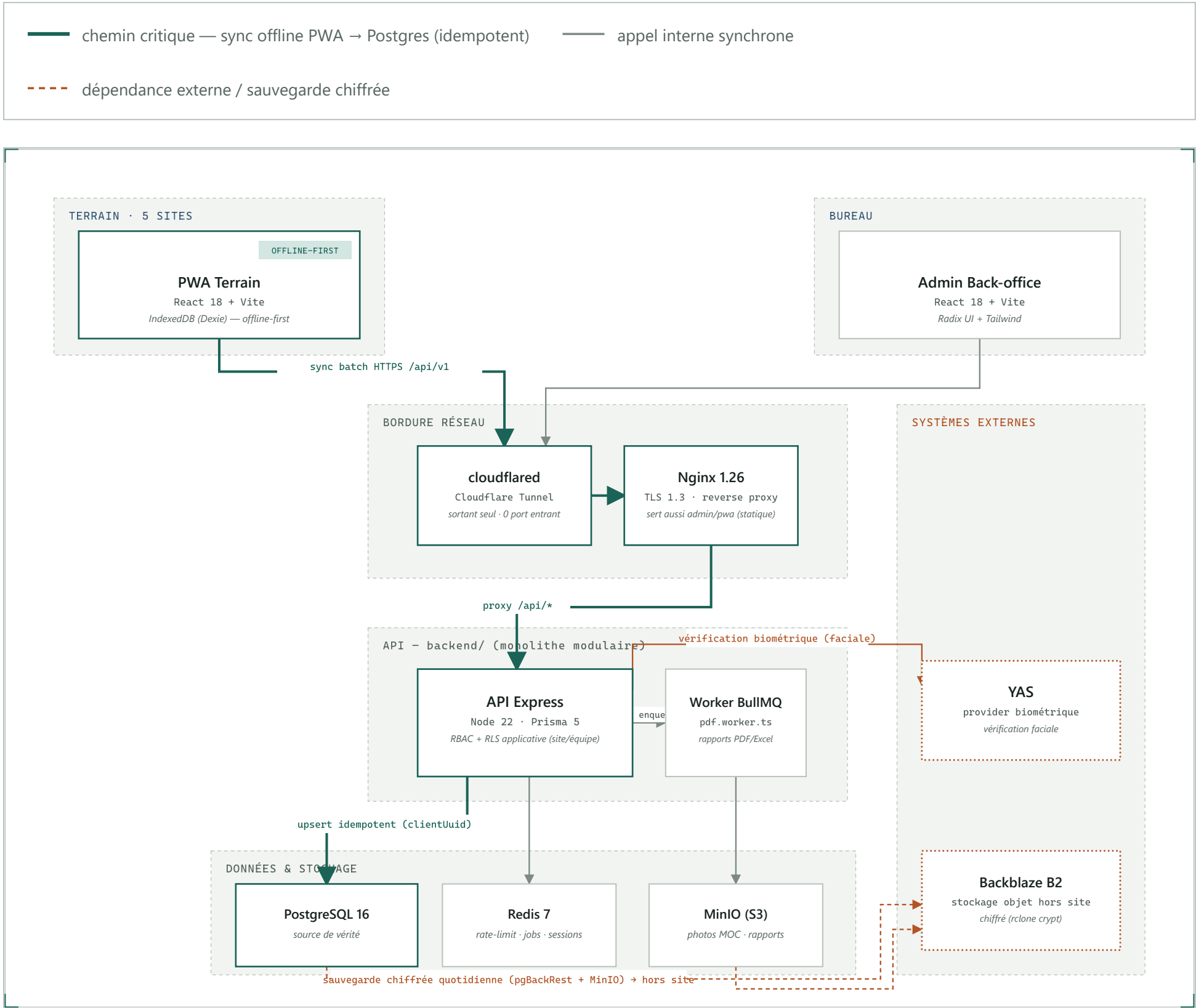


# Architecture cible

Vue conteneurs (C4 — niveau 2) de la production on-premise à Antananarivo : deux fronts, une API monolithique, trois magasins de données, deux dépendances externes.

|             |                          |
|-------------|--------------------------|
| Version     | 1.2                      |
| Périmètre   | 5 sites                  |
| Hébergement | on-premise, Antananarivo |
| Prestataire | Etech                    |
| MàJ source  | 2026-08-13               |
| Diffusion   | Confidentiel             |

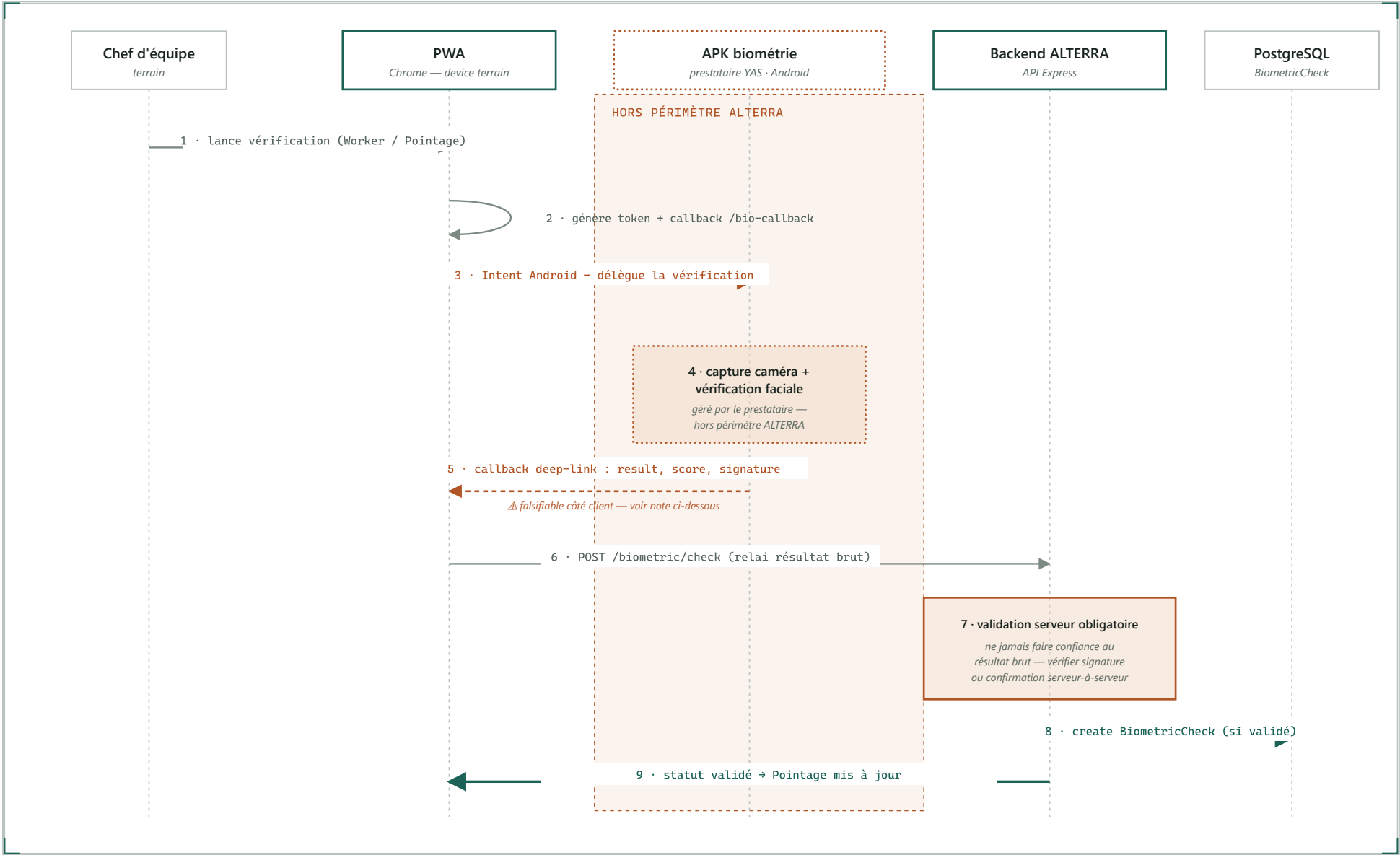


Vue conteneurs C4 (niveau 2). Le trait teal marque le chemin critique de la plateforme — saisie offline sur la PWA, resynchronisée par lot dès retour réseau, upsert idempotent en base via `clientId`. Les traits ocre en pointillés marquent tout ce qui sort du périmètre on-premise ALTERRA : le provider biométrique YAS et la sauvegarde chiffrée hors site (payante, ≈1 USD/mois).

|                                                                                                                                                                                                                                                                              |                                                                                                                                                                                                                               |                                                                                                                                                                                                                                                 |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p><b>Pourquoi un monolithe</b></p> <p>Une seule API Express/Prisma, pas de microservices : périmètre de 5 sites, quelques centaines de MOC, une à deux dizaines d'utilisateurs simultanés. La complexité vient de la connectivité terrain intermittente, pas du volume.</p> | <p><b>Pourquoi un tunnel sortant</b></p> <p>Cloudflare Tunnel évite tout port entrant côté ALTERRA : pas d'IP fixe à gérer, TLS et anti-DDoS inclus, coût nul (offre gratuite). Domaine alterra.mg à la charge d'ALTERRA.</p> | <p><b>Pourquoi l'idempotence par clientId</b></p> <p>Généré côté PWA à la saisie, rejoué sans créer de doublon en cas de resynchronisation après coupure. C'est le mécanisme qui rend le mode offline sûr — pas un détail d'implémentation.</p> |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

# Flux applicatif : Intent APK

Décision du 6 août 2026 : la vérification faciale ne passe plus par un appel REST cloud depuis le backend, mais par une **APK Android** fournie par le prestataire de YAS, invoquée depuis la PWA via un **Intent / deep-link** — la PWA cède le focus à l'APK, qui le lui rend avec un résultat.



Séquence proposée pour la vérification biométrique V2 (source : docs/cadrage/biometrie-v2-intent-apk.md , 6 août 2026). La PWA ne peut pas appeler l'APK comme une librairie — elle lui cède le focus via un Intent Android et le récupère par callback deep-link (pattern OAuth mobile / mobile money). Teal = flux interne ALTERRA maîtrisé. Ocre = hors du périmètre ALTERRA ou point de vigilance sécurité non résolu.

### ⚠ Le résultat de callback (étape 5) est falsifiable côté client

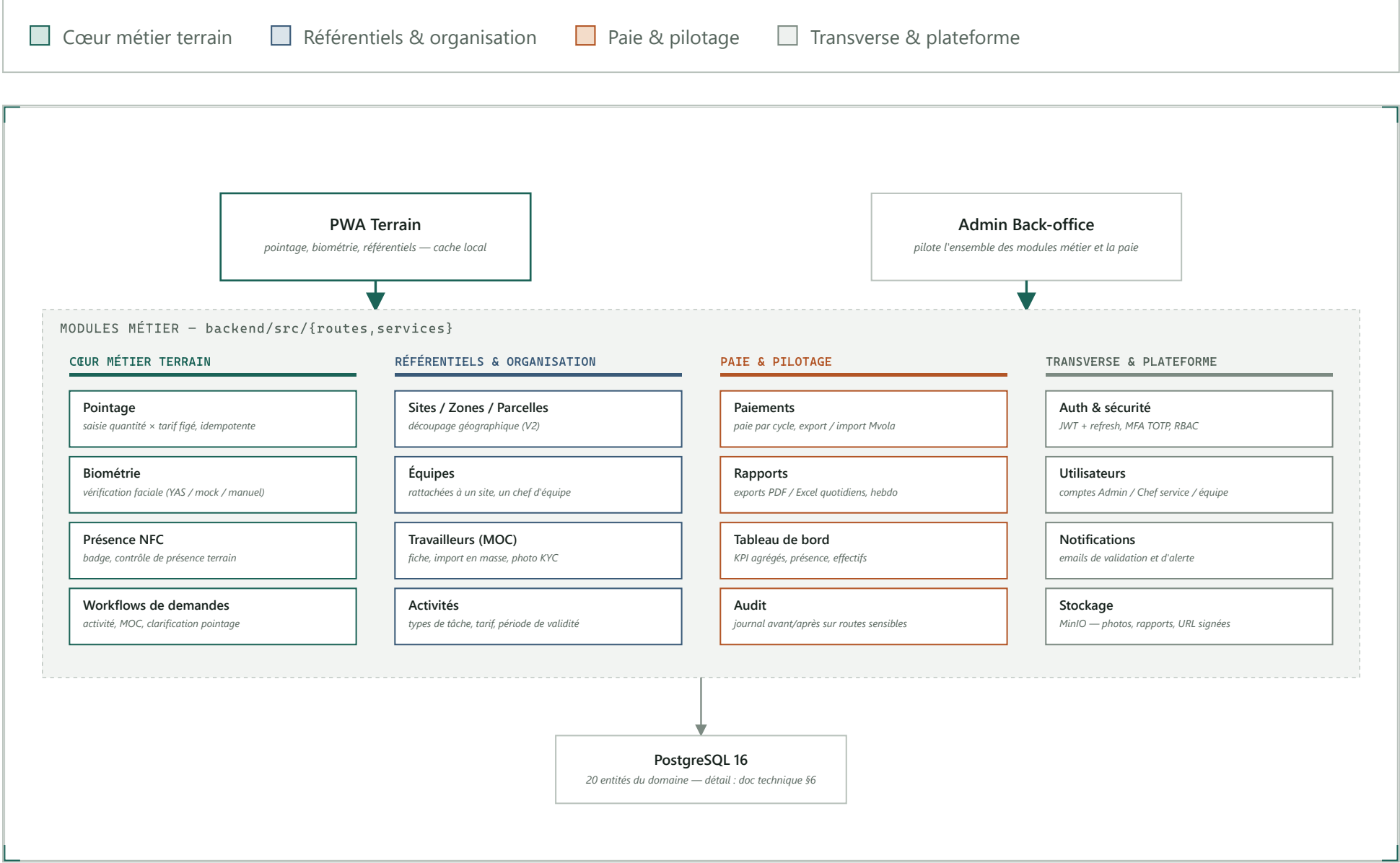
Le retour transite par des query params modifiables par l'utilisateur (result=KO → result=OK sans vraie vérification). Le backend ne doit **jamais** faire confiance au résultat brut — d'où l'étape 7, obligatoire, non négociable. Reste à trancher avec YAS : signature cryptographique vérifiable côté serveur, et/ou rappel serveur-à-serveur avant de valider le BiometricCheck. Sans ça, ce flux ouvre une faille de contournement du contrôle biométrique.

**Questions ouvertes avant implémentation** — ce flux n'est pas intégré à l'architecture cible tant que YAS n'a pas répondu aux deux premières.

| ID | QUESTION                                                                                                                                                                                  | RESPONSABLE         |
|----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------|
| 01 | Format exact de l'intent (package name réel, scheme, paramètres attendus) — doc technique YAS à obtenir                                                                                   | YAS                 |
| 02 | Le retour de callback est-il signé/vérifiable côté serveur ? Quel mécanisme ?                                                                                                             | YAS                 |
| 03 | APK pré-requise/pré-installée sur les devices terrain, ou installation à la volée si absente ?                                                                                            | YAS + Admin ALTERRA |
| 04 | Comportement si connectivité coupée pendant le flux intent (PWA online-only désormais) ?                                                                                                  | YAS                 |
| 05 | La photo est-elle toujours non stockée après envoi, sachant que la capture est faite par l'APK ?<br>Répondu — oui, confirmé : comportement identique à la V1, aucun stockage après envoi. | YAS + RGPD          |

# Carte des modules métier

La vue conteneurs (§1) montre les serveurs et protocoles. Celle-ci montre autre chose : comment le backend découpe le métier en **16 modules** (backend/src/{routes, services}), qui les consomme, et le processus quotidien qu'ils font tourner, du pointage terrain à la clôture.



16 modules backend, chacun avec ses propres `routes/` (parsing + validation Zod) et `services/` (logique métier testable indépendamment d'Express) — cf. doc technique §5.1. Le regroupement en quatre familles est une lecture métier de ce découpage, pas une couche supplémentaire dans le code : dans `backend/src/`, les 16 modules restent des dossiers indépendants au même niveau.

Routes minces, logique dans services/

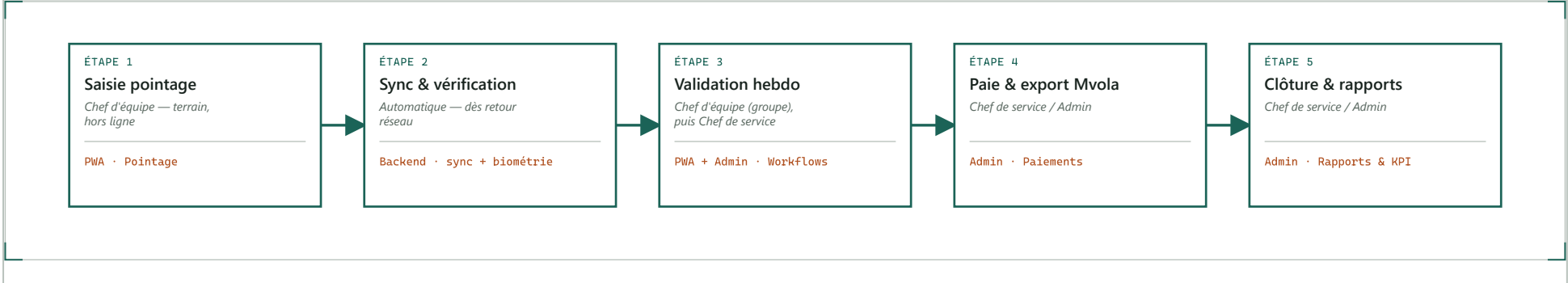
Chaque module suit la même règle : le routeur ne fait que parser/valider/répondre, toute la logique métier vit dans `services/<domaine>/` — testable sans monter Express. C'est ce qui permet de lire ce découpage comme une vraie architecture applicative, pas juste une liste de fichiers.

Pourquoi ces quatre familles

Le code ne connaît que 16 modules à plat ; le regroupement (cœur métier / référentiels / paie / transverse) reflète qui déclenche quoi au quotidien — utile pour discuter périmètre fonctionnel sans lire 16 dossiers un par un.

## Processus métier : du pointage à la clôture

Le fil rouge du produit (PRODUCT.md) : cinq étapes, portées chacune par un rôle et un module précis.



Processus séquentiel — l'ordre porte du sens, d'où la numérotation. Le pointage n'est jamais recalculé après coup : le tarif est figé à la saisie ( `unitRateSnapshot` ), et une correction de paie garde la valeur d'origine ( `Payment.originalAmount` ) pour rester auditable.

ALTERRA – document d'architecture · Etech (prestataire) · Confidentiel – diffusion restreinte · schéma 1 dérivé de docs/architecture-technique.md §2.1, schéma 2 de docs/cadrage/biometrie-v2-intent-apk.md, schémas 3-4 de docs/architecture-technique.md §5-6 et backend/src/{routes, services} · Antananarivo, Madagascar